# Git & GitHub プルリクエスト（PR）体験課題

## 🌟 はじめに：なぜこの課題をやるの？

本リポジトリへようこそ！ここは、実際の開発現場やオープンソースプロジェクトで毎日使われている**「リポジトリのFork」から「Pull Request（PR）の作成」**までの基本フローを一通り体験するための練習用リポジトリ（Playground）です。

個人での開発とは違い、チーム開発では「他人のコードを手元に持ってくる」「自分の変更を分かりやすく提案する」というコミュニケーションのプロセスが発生します。最初は誰でも緊張するものですが、本課題を通してその流れを体で覚えていきましょう！

> 🔰 **失敗しても大丈夫！ここは安全な「遊び場」です** 本課題の提出内容は評価対象にはなりません。間違えて変なブランチを作ってしまったり、エラーを出してしまっても、他の人のコードやメインシステムを壊すことは絶対にありません。 納得がいくまで、**何度でも失敗し、何度でもPRを作成して**、ご自身のスキルアップの場として自由に使い倒してください。PR提出後は、運営チームが優しくレビュー（確認）させていただきます！

---

## 🎯 本課題のポイント

- **選べる2つのコース：** 課題は「1. チャレンジコース（自力で調べる編）」と「2. ハンズオンコース（手順に沿って進める編）」の2部構成です。ご自身の現在のスキルやその日のモチベーションと相談して、好きな方を選んで挑戦してください！
- **正解は一つではありません：** GitやGitHubの操作には、コマンドライン（黒い画面）を使う方法や、VS Codeなどのツールを使う方法など、様々なアプローチがあります。決まった一つの正解はないので、色々と試しながらご自身のやりやすい方法を見つけてみてください。
- **何度でも挑戦OK：** 1度PRを出して承認された後でも、「別のやり方を試してみよう」「違う言語でファイルを追加してみよう」といった形でおかわり挑戦していただくのも大歓迎です。

## 👤 対象となる方

- GitやGitHubに初めて本格的に触れる方
- 個人開発で `add` や `commit` は使っているが、チーム開発（PRを出すフロー）は未経験の方
- エラーが怖くて、気兼ねなくGitコマンドを練習できる「砂場」が欲しい方

## 🛠️ この課題で身につくスキル

- 既存のプロジェクトを安全に分岐させる **「Fork（フォーク）」** の概念
- 自分の作業スペースを作る **「Branch（ブランチ）」** の切り方
- 自分の書いたコードをプロジェクトの大元に提案する **「Pull Request（PR）」** の作成とコミュニケーション

## 🔑 事前準備：毎回パスワードを打つ手間をなくす「魔法の鍵（SSH）」設定

GitHubにコードを送る（Push）たびに、毎回パスワードを聞かれるのはとても面倒ですよね。 そこで作業を始める前に、**「SSH鍵（エスエスエイチかぎ）」**というものを設定して、パソコンとGitHubを安全かつ自動で繋ぐ準備をしましょう！

> 💡 **SSH鍵ってなに？** 「パソコン内に置いておく自分専用の合鍵（秘密鍵）」と、「GitHubに渡しておく南京錠（公開鍵）」のペアのことです。これを設定すると、GitHubが「あ、指定の合鍵を持ってるパソコンからのアクセスだね！」と自動で認識してくれるようになります。

### Step A：ターミナルで鍵のペアを作る

ターミナルを開き、以下のコマンドを入力します。

```
# 「自分のメールアドレス」をあなたのGitHub登録メールアドレスに変えて実行
ssh-keygen -t ed25519 -C "自分のメールアドレス"
```

途中で `Enter file in which to save the key...` や `Enter passphrase...` と何度か英語で聞かれますが、**すべて何も入力せずに「Enterキー」を3回ポンポンポンと押せばOK**です！

### Step B：作った南京錠（公開鍵）の文字をコピーする

次に、GitHubに渡すための公開鍵（文字の羅列）を表示します。

```
cat ~/.ssh/id_ed25519.pub
```

画面に `ssh-ed25519 AAAA...` から始まる長い文字列が表示されるので、それを**最初から最後まで全てマウスで選択してコピー**してください。

### Step C：GitHubに南京錠（公開鍵）を渡す

1. ブラウザでGitHubを開き、右上の自分のアイコンをクリック ＞ **[Settings]** を開きます。
2. 左のメニューから **[SSH and GPG keys]** をクリックします。
3. 緑色の **[New SSH key]** ボタンを押します。
4. **Title**: 自分が分かりやすい名前（例: `My PC` など）
5. **Key**: さきほど Step B でコピーした長い文字列を貼り付けます。
6. **[Add SSH key]** を押して保存完了です！

### Step D：接続テスト！

最後に、正しく繋がったかテストしてみましょう。ターミナルで以下を実行します。

```
ssh -T git@github.com
```

> 💡 **よくある質問：接続テストで長文の英語が出た！** `Are you sure you want to continue connecting (yes/no/[fingerprint])?` というメッセージが出た場合、**設定は順調に進んでいます！**（大正解です🎉）
>
> これは「初めて接続する相手（GitHub）だけど、本当に通信していい？」というパソコンからの安全確認メッセージです。 そのままターミナルに小文字で **`yes`** と入力して **Enterキー** を押してください。

その後、**`Hi (あなたのユーザー名)! You've successfully authenticated...`** と表示されれば設定大成功です！🎉

これで準備は完璧です！早速課題に進みましょう。

# 課題1：チャレンジコース（推奨：まずはAIを使って自力でやってみよう！）

この課題は、経験問わず**できるだけ全員に挑戦していただきたい**メインコースです。 具体的なGitコマンドはあえて記載していません。分からない手順やエラーが出た時は、「ChatGPT」や「Gemini」などの**生成AIを活用して、自分の力で調べながら**進めてみてください！

実際の開発現場でも「AIに聞きながら自己解決する力」は非常に重要です。ぜひゲーム感覚でトライしてみましょう。

## 【ミッション】

指定のリポジトリをForkし、あなた専用のブランチを作成して、「Hello World」を出力するプログラムを追加してPRを出してください。

## 【要件】

1. 以下のリポジトリにアクセスし、ご自身のGitHubアカウントに **Fork** する
    - URL: https://github.com/yasuo-kouki/Hack-Playground
2. Forkしたご自身のリポジトリをローカルに **Clone** する
    - URL例: `https://github.com/<あなたのユーザー名>/Hack-Playground.git`
3. メインのブランチ（`main` など）から、自分用の新しい作業ブランチを作成して切り替える（ブランチ名は自由、例: `feature/yourname-helloworld`）
4. 任意のプログラミング言語で「Hello World」を出力する簡単なコードを作成する。
    - ※**重要：ファイル名には必ず自分の名前を含めてください**（例：`hello_taro.py`, `hello_hanako.js`, `hello_kenji.txt` など）。
5. 変更をステージングし、適切なコミットメッセージをつけて **Commit** する
6. 作成したブランチをリモートリポジトリ（`origin`）に **Push** する
7. GitHubの画面上から、ご自身のブランチから大元のメインブランチへ向けた **Pull Request（PR）** を作成する

# 課題2：ハンズオンコース（どうしても分からない時の解答編）

AIを使っても途中でつまずいてしまった方や、どうしても1ステップずつ答えを確認しながら進めたい方のための解答編（フォールバック）です。 以下の手順に沿って一緒に進めていきましょう。

> 💡 **ワンポイントアドバイス：** 作業中、「今どういう状態だっけ？」と不安になったら、いつでも `git status` や `git branch` というコマンドを実行してください。Gitが現在の状況を親切に教えてくれます。

## Step 1: リポジトリを自分のアカウントにコピーする（Fork）

まずは、指定の「大元のリポジトリ」を自分のGitHubアカウントにコピー（フォーク）します。

1. ブラウザで https://github.com/yasuo-kouki/Hack-Playground にアクセスします。
2. 画面右上の「**Fork**」ボタン（分岐しているアイコン）をクリックします。
3. 「Create fork」画面が出たら、そのまま「**Create fork**」緑色のボタンを押して自分のアカウントにリポジトリを作成します。

## Step 2: リポジトリをローカルに持ってくる（Clone）

次に、Forkして自分のアカウントにできたリポジトリを、あなたのパソコン（ローカル）にダウンロードします。ターミナル（Windowsの場合はGit BashやPowerShellなど）を開いて実行します。

```
# 「your-username」の部分はご自身のGitHubユーザー名に変えてください
git clone https://github.com/your-username/Hack-Playground.git
(https://github.com/your-username/Hack-Playground.git)

# クローンしたフォルダの中に移動します
cd Hack-Playground
```

## Step 3: 現在のブランチを確認する（Branch）

今自分がどのブランチ（作業の分岐）にいるかを確認します。最初はメインのブランチにいるはずです。

```
git branch
```

**【実行結果のイメージ】**

```
* main
```

※ `*` がついていて緑色になっているのが、現在あなたがいるブランチです。

## Step 4: 自分用の作業ブランチを作成して切り替える（Checkout）

他の人の作業とぶつからないように、自分専用の作業スペース（ブランチ）を作って移動します。

```
# 「your-name」の部分は自分の名前（ローマ字）に変えてください
# （-b は「新しいブランチを作って、そこに移動する」という便利なオプションです！）
git checkout -b feature/your-name

# ブランチが正しく作成され、切り替わったか確認します
git branch
```

**【実行結果のイメージ】**

```
  main
* feature/your-name
```

※ `*` が新しく作成したご自身のブランチに移動していれば成功です！

### Step 5: コードを書く（※重要：ファイル名に名前を入れる）

お好きな言語で「Hello World」と出力するプログラムファイルを作成してください。テキストエディタ（VS Codeなど）で作成して保存します。

※誰が作ったファイルか分かるように、**ファイル名には必ず自分の名前を含めてください**（例：`hello_taro.py`, `hello_hanako.js`, `hello_kenji.txt` など）。

### Step 6: ファイルをGitの記録対象に追加する（Add & Status）

ファイルを作成しただけでは、Gitはまだその変更を記録してくれません。「このファイルを次の記録（コミット）に含めてね」とGitに教えるのが `git add` コマンドです。これを「ステージング」と呼びます。

```
# ファイル名を指定して追加します（作成したファイル名に合わせてください）
git add hello_taro.py
```

🌟 **ここで大活躍するのが `git status` です！** `git add` が本当に成功したか、今どんな状態なのか不安に思うことはありませんか？そんな時はすかさず `git status` を叩いてみましょう。Gitが今の状態を分かりやすく教えてくれる、開発者の最強の味方です。

```
git status
```

**【実行結果のイメージ】**

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   hello_taro.py
```

追加したファイル名が**緑色**で表示されていれば、「無事に記録の準備（ステージング）が整っているよ！」というGitからの合図です。作業の合間に息をするように `git status` を打つ癖をつけると、Gitの仕組みがグッと理解しやすくなります。

### Step 7: 変更を記録する（Commit）

`git add` で準備した内容に、「どんな作業をしたか」のメッセージを添えて変更を確定させます。

```
git commit -m "Hello Worldプログラム（自分の名前）を追加"
```

**【実行結果のイメージ】**

```
[feature/your-name a1b2c3d] Hello Worldプログラム（自分の名前）を追加
 1 file changed, 1 insertion(+)
 create mode 100644 hello_taro.py
```

### Step 8: 自分の作業をGitHubへ送る（Push）

ローカル（自分のPC）での記録が終わったので、その内容を自分のGitHub（Fork先のリポジトリ）へ送信します。誤ったブランチを送ってしまわないよう、Pushする前にもう一度現在のブランチを確認しましょう。

```
# 現在自分がどのブランチにいるか最終確認します
git branch

# 自分が作成したブランチにいること（* がついていること）を確認したらPushします
# 「feature/your-name」の部分はご自身のブランチ名にしてください
git push origin feature/your-name
```

### Step 9: Pull Request（PR）を作成する

いよいよ最後のステップです！あなたの変更を大元のリポジトリに取り込んでもらうための申請（PR）を出します。

1. ブラウザで **ご自身がForkしたリポジトリ**（`https://github.com/your-username/Hack-Playground`）にアクセスします。
2. 画面上部に「**Compare & pull request**」という緑色のボタンが出現しているので、クリックします。
3. PRの作成画面になります。以下のように記入してみましょう。
   - **Title（タイトル）：**`[自分の名前] Hello Worldの追加`
   - **Leave a comment（説明文）：**`Hello Worldを出力するプログラムを追加しました。レビューをよろしくお願いします。`
4. 内容を確認し、右下の「**Create pull request**」緑色のボタンをクリックすれば完了です！お疲れ様でした！

🎉 **PR提出後について** PRを作成した後は、運営チームが内容を確認（レビュー）させていただきます。無事に承認（Approve）されるとメインブランチへ統合（Merge）されますので、結果を楽しみにお待ちください！

---

## 💡 おまけTips：作業中に他の人の変更（最新のmain）を取り込む方法

開発現場では、自分が作業している間に他の人がmainブランチを更新することがよくあります。その際、自分のブランチに最新のmainを安全に取り込む（同期する）には、自分が作業しているブランチにいる状態で以下を実行します。

```
# 1. まずは最新の状態をリモートから安全にダウンロード
git fetch origin

# 2. リモートのmainブランチ（origin/main）を自分のブランチに結合する
git merge origin/main
```

---

## 参考サイト

GitやGitHubの操作で迷った時は、AIへの質問とあわせて以下のサイトも参考にしてみてください。

- **初心者向けGit/GitHubの使い方 - Qiita** GitとGitHubの基本的な概念や操作方法について分かりやすくまとめられているQiitaの記事です。
- **【Git入門】pull requestのやり方（作業用ブランチをmainブランチにマージする） - Zenn** Pull Request（PR）の出し方からマージまでの手順が、画面のスクリーンショット付きで丁寧に解説されています。
- **【Git　練習サイト】** GitHubの使い方を体系的に学べるサイトです！自分はこれで学びました！